Mach-O

Mach-O, short for Mach object file format, is a file format for executables, object code, shared libraries, dynamically loaded code, and core dumps. It was developed to replace the a.out format.

Mach-O is used by some systems based on the Mach kernel. NeXTSTEP, macOS, and iOS are examples of systems that use this format for native executables, libraries and object code.

Mach-O

Filename extension	none, `.o`, `.dylib`, `.bundle`
Uniform Type Identifier (UTI)	com.apple.mach-o-binary
Developed by	Carnegie Mellon University, Apple Inc.
Type of format	Binary, executable, object, shared libraries, core dump
Container for	ARM, SPARC, PA-RISC, PowerPC and x86 executable code, memory image dumps

Mach-O file layout

Each Mach-O file is made up of one Mach-O header, followed by a series of load commands, followed by one or more segments, each of which contains between 0 and 255 sections. Mach-O uses the REL relocation format to handle references to symbols. When looking up symbols Mach-O uses a two-level namespace that encodes each symbol into an 'object/symbol name' pair that is then linearly searched for, first by the object and then the symbol name.[1]

The basic structure—a list of variable-length "load commands" that reference pages of data elsewhere in the file[2]—was also used in the executable file format for Accent. The Accent file format was in turn, based on an idea from Spice Lisp.

All multi-byte values in all data structures are written in the byte order of the host for which the code was produced.[3]

Mach-O header

Mach-O File header[4]

Offset	Bytes	Description
0	4	Magic number
4	4	CPU type
8	4	CPU subtype
12	4	File type
16	4	Number of load commands
20	4	Size of load commands
24	4	Flags
28	4	Reserved (64-bit only)

The magic number for 32-bit code is `0xfeedface` while the magic number for 64-bit architectures is `0xfeedfacf`.

The reserved value is only present in 64-bit Mach-O files. It is reserved for future use or extension of the 64-bit header.

The CPU type indicates the instruction set architecture for the code. If the file is for the 64-bit version of the instruction set architecture, the CPU type value has the `0x01000000` bit set.

The CPU type values are as follows:[5]

CPU type

Value	CPU Type
0x00000001	VAX
0x00000002	ROMP
0x00000004	NS32032
0x00000005	NS32332
0x00000006	MC680x0
0x00000007	x86
0x00000008	MIPS
0x00000009	NS32352
0x0000000A	MC98000
0x0000000B	HP-PA
0x0000000C	ARM
0x0000000D	MC88000
0x0000000E	SPARC
0x0000000F	i860 (big-endian)
0x00000010	i860 (little-endian)
0x00000011	RS/6000
0x00000012	PowerPC

Each CPU type has a set of CPU subtype values, indicating a particular model of that CPU type for which the code is intended. Newer models of a CPU type may support instructions, or other features, not supported by older CPU models, so that code compiled or written for a newer model might contain instructions that are illegal instructions on an older model, causing that code to trap or otherwise fail to operate correctly when run on an older model. Code intended for an older model will run on newer models without problems.

If the CPU type is ARM then the subtypes are as follows:[5]

CPU subtype ARM

Value	CPU version
0x00000000	All ARM processors.
0x00000001	Optimized for ARM-A500 ARCH or newer.
0x00000002	Optimized for ARM-A500 or newer.
0x00000003	Optimized for ARM-A440 or newer.
0x00000004	Optimized for ARM-M4 or newer.
0x00000005	Optimized for ARM-V4T or newer.
0x00000006	Optimized for ARM-V6 or newer.
0x00000007	Optimized for ARM-V5TEJ or newer.
0x00000008	Optimized for ARM-XSCALE or newer.
0x00000009	Optimized for ARM-V7 or newer.
0x0000000A	Optimized for ARM-V7F (Cortex A9) or newer.
0x0000000B	Optimized for ARM-V7S (Swift) or newer.
0x0000000C	Optimized for ARM-V7K (Kirkwood40) or newer.
0x0000000D	Optimized for ARM-V8 or newer.
0x0000000E	Optimized for ARM-V6M or newer.
0x0000000F	Optimized for ARM-V7M or newer.
0x00000010	Optimized for ARM-V7EM or newer.

If the CPU type is x86 then the subtypes are as follows:[5]

CPU subtype x86

Value	CPU version
0x00000003	All x86 processors.
0x00000004	Optimized for 486 or newer.
0x00000084	Optimized for 486SX or newer.
0x00000056	Optimized for Pentium M5 or newer.
0x00000067	Optimized for Celeron or newer.
0x00000077	Optimized for Celeron Mobile.
0x00000008	Optimized for Pentium 3 or newer.
0x00000018	Optimized for Pentium 3-M or newer.
0x00000028	Optimized for Pentium 3-XEON or newer.
0x0000000A	Optimized for Pentium-4 or newer.
0x0000000B	Optimized for Itanium or newer.
0x0000001B	Optimized for Itanium-2 or newer.
0x0000000C	Optimized for XEON or newer.
0x0000001C	Optimized for XEON-MP or newer.

After the subtype value is the file type value.

File type

Value	Description
0x00000001	Relocatable object file.
0x00000002	Demand paged executable file.
0x00000003	Fixed VM shared library file.
0x00000004	Core file.
0x00000005	Preloaded executable file.
0x00000006	Dynamically bound shared library file.
0x00000007	Dynamic link editor.
0x00000008	Dynamically bound bundle file.
0x00000009	Shared library stub for static linking only, no section contents.
0x0000000A	Companion file with only debug sections.
0x0000000B	x86_64 kexts.
0x0000000C	a file composed of other Mach-Os to be run in the same userspace sharing a single linkedit.

After the file type value is the number of load commands and the total number of bytes the load commands are after the Mach-O header, then a 32-bit flag with the following possible settings.

Flag Settings

Flag in left shift	Flag in Binary	Description
1<<0	0000_0000_0000_0000_0000_0000_0000_0001	The object file has no undefined references.
1<<1	0000_0000_0000_0000_0000_0000_0000_0010	The object file is the output of an incremental link against a base file and can't be link edited again.
1<<2	0000_0000_0000_0000_0000_0000_0000_0100	The object file is input for the dynamic linker and can't be statically link edited again.
1<<3	0000_0000_0000_0000_0000_0000_0000_1000	The object file's undefined references are bound by the dynamic linker when loaded.
1<<4	0000_0000_0000_0000_0000_0000_0001_0000	The file has its dynamic undefined references prebound.
1<<5	0000_0000_0000_0000_0000_0000_0010_0000	The file has its read-only and read-write segments split.
1<<6	0000_0000_0000_0000_0000_0000_0100_0000	The shared library init routine is to be run lazily via catching memory faults to its writeable segments (obsolete).
1<<7	0000_0000_0000_0000_0000_0000_1000_0000	The image is using two-level name space bindings.
1<<8	0000_0000_0000_0000_0000_0001_0000_0000	The executable is forcing all images to use flat name space bindings.
1<<9	0000_0000_0000_0000_0000_0010_0000_0000	This umbrella guarantees no multiple definitions of symbols in its sub-images so the two-level namespace hints can always be used.
1<<10	0000_0000_0000_0000_0000_0100_0000_0000	Do not have dyld notify the prebinding agent about this executable.
1<<11	0000_0000_0000_0000_0000_1000_0000_0000	The binary is not prebound but can have its prebinding redone. only used when MH_PREBOUND is not set.
1<<12	0000_0000_0000_0000_0001_0000_0000_0000	Indicates that this binary binds to all two-level namespace modules of its dependent libraries.
1<<13	0000_0000_0000_0000_0010_0000_0000_0000	Safe to divide up the sections into sub-sections via symbols for dead code stripping.
1<<14	0000_0000_0000_0000_0100_0000_0000_0000	The binary has been canonicalized via the un-prebind operation.
1<<15	0000_0000_0000_0000_1000_0000_0000_0000	The final linked image contains external weak symbols.
1<<16	0000_0000_0000_0001_0000_0000_0000_0000	The final linked image uses weak symbols.
1<<17	0000_0000_0000_0010_0000_0000_0000_0000	When this bit is set, all stacks in the task will be given stack execution privilege.
1<<18	0000_0000_0000_0100_0000_0000_0000_0000	When this bit is set, the binary declares it is safe for use in processes with uid zero.
1<<19	0000_0000_0000_1000_0000_0000_0000_0000	When this bit is set, the binary declares it is safe for use in processes when UGID is true.
1<<20	0000_0000_0001_0000_0000_0000_0000_0000	When this bit is set on a dylib, the static linker does not need to examine dependent dylibs to see if any are re-exported.

1<<21	0000_0000_0010_0000_0000_0000_0000_0000	When this bit is set, the OS will load the main executable at a random address.
1<<22	0000_0000_0100_0000_0000_0000_0000_0000	Only for use on dylibs. When linking against a dylib that has this bit set, the static linker will automatically not create a load command to the dylib if no symbols are being referenced from the dylib.
1<<23	0000_0000_1000_0000_0000_0000_0000_0000	Contains a section of type S_THREAD_LOCAL_VARIABLES.
1<<24	0000_0001_0000_0000_0000_0000_0000_0000	When this bit is set, the OS will run the main executable with a non-executable heap even on platforms (e.g. i386) that don't require it.
1<<25	0000_0010_0000_0000_0000_0000_0000_0000	The code was linked for use in an application.
1<<26	0000_0100_0000_0000_0000_0000_0000_0000	The external symbols listed in the nlist symbol table do not include all the symbols listed in the dyld info.
1<<27	0000_1000_0000_0000_0000_0000_0000_0000	Allow LC_MIN_VERSION_MACOS and LC_BUILD_VERSION load commands with the platforms macOS, macCatalyst, iOSSimulator, tvOSSimulator and watchOSSimulator.
1<<31	1000_0000_0000_0000_0000_0000_0000_0000	Only for use on dylibs. When this bit is set, the dylib is part of the dyld shared cache, rather than loose in the filesystem.
----	0xxx_0000_0000_0000_0000_0000_0000_0000	The digits marked with "x" have no use, and are reserved for future use.

Multiple binary digits can be set to one in the flags to identify any information or settings that apply to the binary.

Now the load commands are read as we have reached the end of the Mach-O header.

Multi-architecture binaries

Multiple Mach-O files can be combined in a multi-architecture binary. This allows a single binary file to contain code to support multiple instruction set architectures, for example for different generations and types of Apple devices, including different processor architectures[6] such as ARM64 and x86-64.[7]

All fields in the universal header are big-endian.[3]

The universal header is in the following form:[8]

Mach-O universal header

Offset	Bytes	Description
0	4	Magic number
4	4	Number of binaries

The magic number in a multi-architecture binary is `0xcafebabe` in big-endian byte order, so the first 4 bytes of the header will always be `0xca 0xfe 0xba 0xbe`, in that order.

The number of binaries is the number of entries that follow the header.

The header is followed by a sequence of entries in the following form:[9]

Mach-O universal file entries

Offset	Bytes	Description
0	4	CPU type
4	4	CPU subtype
8	4	File offset
12	4	Size
16	4	Section alignment (Power of 2)

The sequence of entries is followed by a sequence of Mach-O images. Each entry refers to a Mach-O image.

The CPU type and subtype for an entry must be the same as the CPU type and subtype for the Mach-O image to which the entry refers.

The file offset and size are the offset in the file of the beginning of the Mach-O image, and the size of the Mach-O image, to which the entry refers.

The section alignment is the logarithm, base 2, of the byte alignment in the file required for the Mach-O image to which the entry refers; for example, a value of 14 means that the image must be aligned on a 2^{14}-byte boundary, i.e. a 16384-byte boundary. This is required by tools that modify the multi-architecture binary, in order for them to keep the image properly aligned.

Load commands

The load commands are read immediately after the Mach-O header.

The Mach-O header tells us how many load commands exist after the Mach-O header and the size in bytes to where the load commands end. The size of load commands is used as a redundancy check.

When the last load command is read and the number of bytes for the load commands do not match, or if we go outside the number of bytes for load commands before reaching the last load command, then the file may be corrupted.

Each load command is a sequence of entries in the following form:[10]

Load command

Offset	Bytes	Description
0	4	Command type
4	4	Command size

The load command type identifies what the parameters are in the load command. If a load command starts with `0x80000000` bit set that means the load command is necessary in order to be able to load or run the binary. This allows older Mach-O loaders to skip commands not understood by the loader that are not mandatory for loading the application.

Segment load command

Mach-O binaries that use load command type `0x00000001` use the 32-bit version of the segment load command,[11] while `0x00000019` is used to specify the 64-bit version of the segment load command.,[12]

The segment load command varies if the Mach-O header is 32-bit, or 64-bit. This is because 64-bit processor architecture uses 64-bit addresses while 32-bit architectures use 32-bit addresses.

All virtual RAM addresses are added to a base address to keep applications spaced apart. Each section in a segment load command has a relocation list offset that specifies the offsets in the section that must be adjusted based on the application's base address. The relocations are unnecessary if the application can be placed at its defined RAM address locations such as a base address of zero.

Load command (Segment load32/64)

Offset(32-bit)	Bytes(32-bit)	Offset(64-bit)	Bytes(64-bit)	Description
0	4	0	4	`0x00000001` (Command type 32-bit) `0x00000019` (Command type 64-bit)
4	4	4	4	Command size
8	16	8	16	Segment name
24	4	24	8	Address
28	4	32	8	Address size
32	4	40	8	File offset
36	4	48	8	Size (bytes from file offset)
40	4	56	4	Maximum virtual memory protections
44	4	60	4	Initial virtual memory protections
48	4	64	4	Number of sections
52	4	68	4	Flag32

A segment name cannot be larger than 16 text characters in bytes. The unused characters are `0x00` in value.

The segment command contains the address to write the section in virtual address space plus the application's base address. The number of bytes to write to the address location (Address size).

After the address information is the file offset the segment data is located in the Mach-O binary, and the number of bytes to read from the file.

When the address size is larger than the number of bytes to read from the file, the rest of the bytes in RAM space are set `0x00`.

There is a segment that is called __PAGEZERO, which has a file offset of zero and a size of zero in the file. It has a defined RAM address and size. Since it reads zero bytes from the file it fills the address location with zeros to where the binary is going to be placed in RAM. This segment is necessary to rid the section of any data from a prior application.

When a segment is initially placed in the virtual address space, it is given the CPU access permissions specified by the initial virtual memory protections value. The permissions on a region of the virtual address space may be changed by application or library code with calls to routines such as `mprotect()`; the maximum virtual memory protections limit what permissions may be granted for access to the segment.

Permissions

Permission bit in binary	Description
`00000000000000000000000000000001`	The section allows the CPU to read data from this section (Read setting).
`00000000000000000000000000000010`	The section allows the CPU to write data to this section (Write setting).
`00000000000000000000000000000100`	The section allows the CPU to execute code in this section (Execute setting).
`xxxxxxxxxxxxxxxxxxxxxxxxxxxxx000`	The digits marked with "x" have no use, and are reserved for future use.

Then after the CPU address protection settings is the number of sections that are within this segment that are read after the segments flag settings.

The segment flag settings are as follows:

Segment flag settings.

Flag32 in Binary	Description
`00000000000000000000000000000001`	The file contents for this segment is for the high part of the VM space, the low part is zero filled (for stacks in core files).
`00000000000000000000000000000010`	This segment is the VM that is allocated by a fixed VM library, for overlap checking in the link editor.
`00000000000000000000000000000100`	This segment has nothing that was relocated in it and nothing relocated to it, that is it maybe safely replaced without relocation.
`00000000000000000000000000001000`	This segment is protected. If the segment starts at file offset 0, the first page of the segment is not protected. All other pages of the segment are protected.
`00000000000000000000000000010000`	This segment is made read-only after relocations are applied if needed.
`xxxxxxxxxxxxxxxxxxxxxxxxxxx00000`	The digits marked with "x" have no use, and are reserved for future use.

The number of sections in the segment is a set of entries that are read as follows:

Segment section32/64

Offset(32-bit)	Bytes(32-bit)	Offset(64-bit)	Bytes(64-bit)	Description
0	16	0	16	Section name
16	16	16	16	Segment name
32	4	32	8	Section Address
36	4	40	8	Section size
40	4	48	4	Section file offset
44	4	52	4	Alignment
48	4	56	4	Relocations file offset
52	4	60	4	Number of relocations
56	4	64	4	Flag/Type
60	4	68	4	Reserved1
64	4	72	4	Reserved2
N/A	N/A	76	4	Reserved3 (64-bit only)

The section's segment name must match the segments load command name. The sections entries locate to data in the segment. Each section locates to the relocation entries for adjusting addresses in the section if the application base address is added to anything other than zero.

The section size applies to both the size of the section at its address location and size in the file at its offset location.

The section Flag/Type value is read as follows:

Section flag settings

Flag in binary	Description
`10000000000000000000000000000000xxxxxxxx`	Section contains only true machine instructions
`01000000000000000000000000000000xxxxxxxx`	Section contains coalesced symbols that are not to be in a ranlib table of contents
`00100000000000000000000000000000xxxxxxxx`	Ok to strip static symbols in this section in files with the MH_DYLDLINK flag
`00010000000000000000000000000000xxxxxxxx`	No dead stripping
`00001000000000000000000000000000xxxxxxxx`	Blocks are live if they reference live blocks
`00000100000000000000000000000000xxxxxxxx`	Used with i386 code stubs written on by dyld
`00000010000000000000000000000000xxxxxxxx`	A debug section
`00000000000000000000010000000000xxxxxxxx`	Section contains some machine instructions
`00000000000000000000001000000000xxxxxxxx`	Section has external relocation entries
`00000000000000000000000100000000xxxxxxxx`	Section has local relocation entries

Any of the settings that apply to the section have a binary digit set one. The last eight binary digits is the section type value.

Section type value

Flag in binary	Description
`xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx00000110`	Section with only non-lazy symbol pointers
`xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx00000111`	Section with only lazy symbol pointers
`xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx00001000`	Section with only symbol stubs
`xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx00001100`	Zero fill on demand section (that can be larger than 4 gigabytes)
`xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx00010000`	Section with only lazy symbol pointers to lazy loaded dylibs

The Mach-O loader records the symbol pointer sections and symbol stub sections. They are sequentially used by the indirect symbol table to load in method calls.

The size of each symbol stub is stored in reserved2 value. Each pointer is 32-bit address locations in 32-bit Mach-O and 64-bit address locations in 64-bit Mach-O. Once the section end is reached, we move to the next section while reading the indirect symbol table.

Segment number and section numbers

The segments and sections are located by segment number and section number in the compressed and uncompressed link edit information sections.

A segment value of 3 would mean the offset to the data of the fourth segment load command in the Mach-O file starting from zero up (0,1,2,3 = 4th segment).

Sections are also numbered from sections 1 and up. Section value zero is used in the symbol table for symbols that are not defined in any section (undefined). Such as an method, or data that exist within another binaries symbol table section.

A segment that has 7 sections would mean the last section is 8. Then if the following segment load command has 3 sections they are labelled as sections 9, 10, and 11. A section number of 10 would mean the second segment, section 2.

We would not be able to properly read the symbol table and linking information if we do not store the order the sections are read in and their address/file offset position.

You can easily use file offset without using the RAM addresses and relocations to build a symbol reader and to read the link edit sections and even map method calls or design a disassembler.

If building a Mach-O loader, then you want to dump the sections to the defined RAM addresses plus a base address to keep applications spaced apart so they do not write over one another.

The segment names and section names can be renamed to anything you like and there link will be no problems locating the appropriate sections by section number, or segment number as long as you do not alter the order the segment commands go in.

Link libraries

Link libraries are the same as any other Mach-O binary, just that there is no command that specifies the main entry point at which the program begins.

There are three load commands for loading a link library file.

Load command type `0x0000000C` are for the full file path to the dynamically linked shared library.

Load command type `0x0000000D` are for dynamically linked shared locations from the application's current path.

Load command type `0x00000018` is for a dynamically linked shared library that is allowed to be missing. The symbol names exist in other link libraries and are used if the library is missing meaning all symbols are weak imported.

The link library command is read as follows:

Load command (Link library)

Offset	Bytes	Description
0	4	`0x0000000C` (Command type) `0x0000000D` (Command type) `0x00000018` (Command type)
4	4	Command size
8	4	String offset (always offset 24)
12	4	Time date stamp
16	4	Current version
20	4	Compatible version
24	Command size - 24	File path string

The file path name begins at the string offset, which is always 24. The number of bytes per text character is the remaining bytes in command size. The end of the library file path is identified by a character that is `0x00`. The remaining `0x00` values are used as padding, if any.

Link library ordinal numbers

The library is located by ordinal number in the compressed and uncompressed link edit information sections.

Link libraries are numbered from ordinal 1 and up. The ordinal value zero is used in the symbol table to specify the symbol does not exist as an external symbol in another Mach-O binary.

The link edit information will have no problem locating the appropriate library to read by ordinal number as long as you do not alter the order in which the link library commands go in.

Link library command `0x00000018` should be avoided for performance reasons, as in the case the library is missing, then a search must be performed through all loaded link libraries.

__LINKEDIT Symbol table

Mach-O application files and link libraries both have a symbol table command.

The command is read as follows:

Load command (Symbol table)

Offset	Bytes	Description
0	4	`0x00000002` (Command type)
4	4	Command size (always 24)
8	4	Symbols (file offset relative to Mach-O header)
12	4	Number of Symbols
16	4	String table (file offset relative to Mach-O header)
20	4	String table size

The symbol file offset is the offset relative to the start of the Mach-O header to where the symbol entries begins in the file. The number of symbol entries marks the end of the symbol table.

A symbol has a name offset that should never exceed the string table size. Each symbol name offset is added to the string table file offset which in turn is relative to the start of the Mach-O header. Each symbol name ends with a `0x00` byte value.

The symbol address uses a 32-bit address for 32-bit Mach-O files and a 64-bit address for 64-bit Mach-O files.

Each symbol entry is read as follows:

Symbol32/64

Offset(32-bit)	Bytes(32-bit)	Offset(64-bit)	Bytes(64-bit)	Description
0	4	0	4	Name offset
4	1	4	1	Symbol type
5	1	5	1	Section number 0 to 255
6	2	6	2	Data info (library ordinal number)
8	4	8	8	Symbol address

The symbol name offset is added to the string table offset. The last text character byte is read as `0x00`.

The symbol type value has multiple adjustable sections in binary. The symbol type is read as follows:

Symbol type sections

Binary digits	Description
`???xxxxx`	Local debugging symbols
`xxxx???x`	Symbol address type
`xxx?xxx?`	Symbol visibility setting flags

The digits marked `?` are used for the specified purpose; the digits marked `x` are used for other purposes.

The three first binary digits are symbols that locate to function names relative to compiled machine code instructions and line numbers by address location. This information allows us to generate line numbers to the location your code crashed. Local debugging symbols are only useful when designing the application, but are not needed to run the application.

Symbol address type

Binary value	Description
`xxxx000x`	Symbol undefined
`xxxx001x`	Symbol absolute
`xxxx101x`	Symbol indirect
`xxxx110x`	Symbol prebound undefined
`xxxx111x`	Symbol defined in section number

The following flag settings:

Symbol visibility setting flags

Binary value	Description
`xxx1xxx0`	Private symbol
`xxx0xxx1`	External symbol

External symbols are symbols that have a defined address in the link library and can be copied to an undefined symbol in a Mach-O application. The address location is added to the link library base address.

A private symbol is skipped even if it matches the name of an undefined symbol. A private and external symbol can only be set to an undefined symbol if it is in the same file.

After the symbol type is the section number the symbol exists in. The section number is a byte value (0 to 255). You can add more sections than 255 using segment load commands, but the section numbers are then outside the byte value range used in the symbol entries.

A section number of zero means the symbol is not in any section of the application, the address location of the symbol is zero, and is set as Undefined. A matching External symbol name has to be found in a link library that has the symbol address.

The data info field contains the link library ordinal number that the external symbol can be found in with the matching symbol name. The data info bit field breaks down as follows:

Symbol data info sections

Binary digits	Description
????????xxxxxxxx	Library ordinal number 0 to 255
xxxxxxxx????xxxx	Dynamic loader flag options
xxxxxxxxxxxx????	Address type option

The library ordinal number is set zero if the symbol is an external symbol, or exists in the current file. Only undefined symbols use the data info section to specify a library ordinal number and linker options.

The dynamic loader flag options are as follows:

Dynamic loader flag options

Binary digits	Description
xxxxxxxx0001xxxx	Must be set for any defined symbol that is referenced by dynamic-loader.
xxxxxxxx0010xxxx	Used by the dynamic linker at runtime.
xxxxxxxx0100xxxx	If the dynamic linker cannot find a definition for this symbol, it sets the address of this symbol to 0.
xxxxxxxx1000xxxx	If the static linker or the dynamic linker finds another definition for this symbol, the definition is ignored.

Any of the 4 options that apply can be set.

The address type option values are as follows:

Dynamic loader address options

Binary digits	Description
xxxxxxxxxxxx0000	Non Lazy loaded pointer method call
xxxxxxxxxxxx0001	Lazy loaded pointer method call
xxxxxxxxxxxx0010	Method call defined in this library/program
xxxxxxxxxxxx0011	Private Method call defined in this library/program
xxxxxxxxxxxx0100	Private Non Lazy loaded pointer method call
xxxxxxxxxxxx0101	Private Lazy loaded pointer method call

Only one address type value can be set by value. A Pointer is a value that is read by the program machine code to call a method from another binary file. Private means other programs are not intended to be able to read or call the function/methods other than the binary itself. Lazy means the pointer locates to the dyld_stub_binder which looks for the symbol then calls the method, then replaces the dyld_stub_binder location with the location to the symbol. Any more calls done from machine code in the binary will now locate to the address of the symbol and will not call the dyld_stub_binder.

Symbol table organization

The symbol table entries are all stored in order by type. The first symbols that are read are local debug symbols if any, then private symbols, then external symbols, and finally the undefined symbols that link to another binary symbol table containing the external symbol address in another Mach-O binary.

The symbol table information load command `0x0000000B` always exists if there is a symbol table section in the Mach-O binary. The command tells the linker how many local symbols there are, how many private, how many external, and how many undefined. It also identifies the symbol number they start at. The symbol table information is used before reading the symbol entries by the dynamic linker as it tells the dynamic linker where to start reading the symbols to load in undefined symbols and where to start reading to look for matching external symbols without having to read all the symbol entries.

The order the symbols go in the symbol section should never be altered as each symbol is numbered from zero up. The symbol table information command uses the symbol numbers for the order to load the undefined symbols into the stubs and pointer sections. Altering the order would cause the wrong method to be called during machine code execution.

__LINKEDIT Symbol table information

The symbol table information command is used by the dynamic linker to know where to read the symbol table entries under symbol table command `0x00000002`, for fast lookup of undefined symbols and external symbols while linking.

The command is read as follows:

Load command (Symbol table information)

Offset	Bytes	Description
0	4	`0x0000000B` (Command type)
4	4	Command size (always 80)
8	4	Local symbol index
12	4	Number of local symbols
16	4	External symbols index
20	4	Number of exsternal symbols
24	4	Undefined symbols index
28	4	Number of undefined symbols
32	4	Content table offset
36	4	Number of content table entries
40	4	Module table offset
44	4	Number of module table entries
48	4	Offset to referenced symbol table
52	4	Number of referenced symbol table entries
56	4	Indirect symbol table offset
60	4	Indirect symbol table entries
64	4	Exsternal relocation offset
68	4	Number of exsternal relocation entries
72	4	Local relocation offset
76	4	Number of Local relocation entries

The symbol index is multiplied by 12 for Mach-O 32-bit, or 16 for Mach-O 64-bit plus the symbol table entries offset to find the offset to read the symbol entries by symbol number index.

The local symbol index is zero as it is at the start of the symbol entries. The local symbols are used for debugging information.

Number of local symbols is how many exist after the symbol index.

The same two properties are repeated for external symbols and undefined symbols for fast reading of the symbol table entries.

There is a small index/size gap between local symbols and external symbols if there are private symbols.

Any file offsets that are zero are unused.

Indirect table

The Mach-O loader records the symbol pointer sections and symbol stub sections during the segment load commands. They are sequentially used by the indirect symbol table to load in method calls. Once the section end is reached, we move to the next.

The Indirect symbol table offset locates to a set of 32-bit (4-byte) values that are used as a symbol number index.

The order the symbol index numbers go is the order we write each symbol address one after another in the pointer and stub sections.

The symbol stub section contains machine code instructions with JUMP instructions to the indirect symbol address to call a method/function from another Mach-O binary. The size of each JUMP instruction is based on processor type and is stored in the reserved2 value under the section32/64 of a segment load command.

The pointer sections are 32-bit (4-byte) address values for 32-bit Mach-O binaries and 64-bit (8-byte) address values for 64-bit Mach-O binaries. Pointers are read by machine code and the read value is used as the location to call the method/function rather than containing machine code instructions.

A symbol index number `0x40000000` bit set are absolute methods meaning the pointer locates to the exact address of a method.

A symbol index number `0x80000000` bit set are local methods meaning the pointer itself located to the method and that there is no method name (Local method).

If you are designing a disassembler you can easily map just the symbol name to the offset address of each stub and pointer to show the method or function call taking place without looking for the undefined symbol address location in other Mach-O files.

__LINKEDIT Compressed table

If the compressed link edit table command exists, then the undefined/external symbols in the symbol table are no longer needed. The indirect symbol table and location of the stubs and pointer sections are no longer required.

The indirect symbol table still exists in the case of building backwards compatible Mach-O files that load on newer and older OS versions.

Load command (Compressed link edit table)

Offset	Bytes	Description
0	4	`0x00000022` (Command type)
4	4	Command size (always 48 bytes)
8	4	Rebase file offset
12	4	Rebase size
16	4	Bind file offset
20	4	Bind size
24	4	Weak bind file offset
28	4	Weak bind size
32	4	Lazy bind file offset
36	4	Lazy bind size
40	4	Export file offset
44	4	Export size

Any file offsets that are zero are sections that are unused.

Binding information

The bind, weak bind, and lazy bind sections are read using the same operation code format.

Originally the symbol table would define the address type in the data info field in the symbol table as lazy, weak, or non-lazy.

Weak binding means that if the set library to look in by library ordinal number, and the set symbol name does not exist but exists under a different previously loaded Mach-O file then the symbol location is used from the other Mach-O file.

Lazy means the address that is written located to the dyld_stub_binder, which looks for the symbol then calls the method, then replaces the dyld_stub_binder location with the location to the symbol. Any more calls done from machine code in the binary will now locate to the address of the symbol and will not call the dyld_stub_binder.

The plain old bind section does not do any fancy loading or address tricks. The symbol must exist in the set library ordinal.

A byte value that is `0x1X` sets the link library ordinal number. The hex digit that is X is a 0 to 15 library ordinal number.

A byte value that is `0x20` to `0x2F` sets the link library ordinal number to the value that is read after the operation code.

The byte sequence `0x20 0x84 0x01` set ordinal number 132.

The number value after the operation code is encoded as a LEB128 number. The last 7 binary digits are added together to form a larger number as long as the last binary digit is set one in value. This allows us to encode variable length number values.

A byte value that is `0x4X` sets the symbol name. The hex digit marked X sets the flag setting.

Flag setting 8 means the method is weak imported. Flag setting 1 means the method is non weak imported.

The byte sequence `0x48 0x45 0x78 0x61 0x6D 0x70 0x6C 0x65 0x00` sets the symbol name Example. The last text character byte is `0x00`. It is also weak imported, meaning it can be replaced if another exportable symbol is found with the same name.

A byte value `0x7X` sets the current location. The hex digit marked X is the selected segment 0 to 15. After the operation code is the added offset as a LEB128 number to the segment offset.

The byte sequence `0x72 0x8C 0x01` sets the location to the third segment load command address and adds 140 to the address.

Operation code `0x90` to `0x9F` binds the current set location to the set symbol name and library ordinal. Increments the current set location by the size 4 bytes for a 32-bit Mach-O binary or increments the set address by 8 for a 64-bit Mach-O binary.

The byte sequence `0x11 0x72 0x8C 0x01 0x48 0x45 0x78 0x61 0x6D 0x70 0x6C 0x65 0x00 0x90 0x48 0x45 0x78 0x61 0x6D 0x70 0x6C 0x65 0x32 0x00 0x90`

Sets link library ordinal 1. Set location to segment number 2, and adds 140 to the current location. Looks for a symbol named Example in the selected library ordinal number. Operation code `0x90` writes the symbol address and increments the current set address. The operation code after that sets the next symbol name to look for a symbol named Example2. Operation code `0x90` writes the symbol address and increments the current set address.

The new format removes the repeated fields in the symbol table and makes the indirect symbol table obsolete.

Application main entry point

A load command starting with type `0x00000028` is used to specify the address location the application begins at.

Load command (Main entry point)

Offset	Bytes	Description
0	4	`0x00000028` (Command type)
4	4	Command size (always 24 bytes)
8	8	Address location
16	8	Stack memory size

If the segments/sections of the program do not have to be relocated to run, then the main entry point is the exact address location. This is only if the application segment addresses are added to an application base address of zero and the sections did not need any relocations.

The main entry point in a Mach-O loader is the program's base address plus the Address location. This is the address at which the CPU is set to begin running machine code instructions.

This replaced the old load command `0x00000005` which varied by CPU type as it stored the state that all the registers should be at before the program starts.

Application UUID number

A load command starting with type `0x0000001B` is used to specify the universally unique identifier (UUID) of the application.

Load command (UUID number)

Offset	Bytes	Description
0	4	`0x0000001B` (Command type)
4	4	Command size (always 24 bytes)
8	16	128-bit UUID

The UUID contains a 128-bit unique random number when the application is compiled that can be used to identify the application file on the internet or in app stores.

Minimum OS version

A load command starting with type `0x00000032` is used to specify the minimum OS version information.

Load command (Minimum OS version)

Offset	Bytes	Description
0	4	`0x00000032` (Command type)
4	4	Command size
8	4	Platform type
12	4	Minimum OS version
16	4	SDK version
20	4	Number of tools used

The Platform type the binary is intended to run on are as follows:

Platform type.

Value	Platform
0x00000001	MacOS
0x00000002	iPhone IOS
0x00000003	Apple TV Box
0x00000004	Apple Watch
0x00000005	Bridge OS
0x00000006	Mac Catalyst
0x00000007	iPhone IOS simulator
0x00000008	Apple TV simulator
0x00000009	Apple watch simulator
0x0000000A	Driver KIT
0x0000000B	Apple Vision Pro
0x0000000C	Apple Vision Pro simulator

The 32-bit version value is read as a 16-bit value and two 8-bit values. A 32-bit version value of `0x000D0200` breaks down as `0x000D` which is 13 in value, then the next 8-bits is `0x02` which is 2 in value, then the last 8-bits is `0x00` which is zero in value giving a version number of 13.2.0v. The SDK version value is read the same way.

The number of tools to create the binary is a set of entries that are read as follows:

Tool type

Offset	Bytes	Description
0	4	Tool type
4	4	Vestion type

The tool type values are as follows:

Tool type value.

Value	Tool type used
0x00000001	CLANG
0x00000002	SWIFT
0x00000003	LD

The version number is read the same as OS version and SDK version.

With the introduction of Mac OS X 10.6 platform the Mach-O file underwent a significant modification that causes binaries compiled on a computer running 10.6 or later to be (by default) executable only on computers running Mac OS X 10.6 or later. The difference stems from load commands that the dynamic linker, in previous Mac OS X versions, does not understand. Another significant change to the Mach-O format is the change in how the Link Edit tables (found in the __LINKEDIT section) function. In 10.6

these new Link Edit tables are compressed by removing unused and unneeded bits of information, however Mac OS X 10.5 and earlier cannot read this new Link Edit table format. To make backwards-compatible executables, the linker flag "-mmacosx-version-min=" can be used.

Other implementations

A Mach-O application can be run on different operating systems or OS as long as a Mach-O binary image exists that matches the core type in your computer. Most desktops are x86, meaning that a Mach-O with an x86 binary will run without problems if you load the sections into memory. If the Mach-O is designed for iPhone, which has an ARM core, then you would need a PC with an ARM core (does not have to be apple silicon ARM) to run it; otherwise, you would have to change ARM encoded instructions to equivalent x86 encoded instructions. The problem of loading and directly executing a Mach-O is undefined symbols that call functions/methods from other Mach-O binaries that do not exist on another operating system. Some symbols can call other equivalent functions in the different operating systems or even call adaptor functions to make other binary function calls behave like the macOS equivalents. The Mach-O files stored on the device can vary between iPhone (iOS), macOS, watchOS, and tvOS. Causing differences in function calls from undefined symbols.

Some versions of NetBSD have had Mach-O support added as part of an implementation of binary compatibility, which allowed some Mac OS 10.3 binaries to be executed.[13][14]

For Linux, a Mach-O loader was written by Shinichiro Hamaji[15] that can load 10.6 binaries. As a more extensive solution based on this loader, the Darling Project aims at providing a complete environment allowing macOS applications to run on Linux.

For the Ruby programming language, the ruby-macho[16] library provides an implementation of a Mach-O binary parser and editor.

See also

- Fat binary
- Universal binary
- Dynamic linker
- Mac transition to Intel processors
- Mac transition to Apple silicon
- Xcode
- ELF
- Comparison of executable file formats

References

1. "OS X ABI Mach-O File Format Reference" (https://web.archive.org/web/20140904004108/https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html). Apple Inc. February 4, 2009. Archived from the original (https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html) on September 4, 2014.

2. Avadis Tevanian, Jr.; Richard F. Rashid; Michael W. Young; David B. Golub; Mary R. Thompson; William Bolosky; Richard Sanzi (June 1987). "A Unix Interface for Shared Memory and Memory Mapped Files Under Mach" (http://citeseer.ist.psu.edu/viewdoc/summary?doi=10.1.1.52.675). *Proceedings of the USENIX Summer Conference*. Phoenix, AZ, USA: USENIX Association}. pp. 53–67.
3. "Data Types" (https://web.archive.org/web/20140904004108mp_/https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html#//apple_ref/doc/uid/20001298-BAJFFCGF). *OS X ABI Mach-O File Format Reference*. Apple Inc. February 4, 2009 [2003]. Archived from the original (https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html#//apple_ref/doc/uid/20001298-BAJFFCGF) on September 4, 2014.
4. loader.h (https://github.com/apple/darwin-xnu/blob/main/EXTERNAL_HEADERS/mach-o/loader.h) on GitHub
5. machine.h (https://github.com/opensource-apple/cctools/blob/master/include/mach/machine.h) on GitHub
6. "Universal Binaries and 32-bit/64-bit PowerPC Binaries" (https://web.archive.org/web/20140904004108/https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html#//apple_ref/doc/uid/20001298-154889). *OS X ABI Mach-O File Format Reference*. Apple Inc. February 4, 2009 [2003]. Archived from the original (https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html#//apple_ref/doc/uid/20001298-154889) on September 4, 2014.
7. "Building a Universal macOS Binary" (https://developer.apple.com/documentation/apple-silicon/building-a-universal-macos-binary). *Apple Developer*.
8. "fat_header" (https://web.archive.org/web/20140904004108/https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html#//apple_ref/c/tag/fat_header). *OS X ABI Mach-O File Format Reference*. Apple Inc. February 4, 2009 [2003]. Archived from the original (https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html#//apple_ref/c/tag/fat_header) on September 4, 2014.
9. "fat_arch" (https://web.archive.org/web/20140904004108/https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html#//apple_ref/c/tag/fat_arch). *OS X ABI Mach-O File Format Reference*. Apple Inc. February 4, 2009 [2003]. Archived from the original (https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html#//apple_ref/c/tag/fat_arch) on September 4, 2014.
10. "Load Command Data Structures" (https://web.archive.org/web/20140904004108mp_/https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html#//apple_ref/doc/uid/20001298-TPXREF114). *OS X ABI Mach-O File Format Reference*. Apple Inc. February 4, 2009 [2003]. Archived from the original (https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html#//apple_ref/doc/uid/20001298-TPXREF114) on September 4, 2014.
11. "segment_command" (https://web.archive.org/web/20140904004108mp_/https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html#//apple_ref/doc/uid/20001298-segment_command). *OS X ABI Mach-O File Format Reference*. Apple Inc. February 4, 2009 [2003]. Archived from the original (https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html#//apple_ref/doc/uid/20001298-segment_command) on September 4, 2014.

12. "segment_command_64" (https://web.archive.org/web/20140904004108mp_/https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html#//apple_ref/doc/uid/20001298-CJBDHJGA). *OS X ABI Mach-O File Format Reference*. Apple Inc. February 4, 2009 [2003]. Archived from the original (https://developer.apple.com/library/mac/documentation/developertools/conceptual/MachORuntime/Reference/reference.html#//apple_ref/doc/uid/20001298-CJBDHJGA) on September 4, 2014.
13. Emmanuel Dreyfus (June 20, 2006). "Mach and Darwin binary compatiblity [*sic*] for NetBSD/powerpc and NetBSD/i386" (http://hcpnet.free.fr/applebsd.html). Retrieved October 18, 2013.
14. Emmanuel Dreyfus (September 2004), *Mac OS X binary compatibility on NetBSD: challenges and implementation* (http://2004.eurobsdcon.org/uploads/media/EBSD04_21.pdf) (PDF)
15. Shinichiro Hamaji, *Mach-O loader for Linux - I wrote...* (http://shinh.skr.jp/slide/ldmac/001.html)
16. William Woodruff (November 15, 2021), *A pure-Ruby library for parsing Mach-O files.* (https://github.com/Homebrew/ruby-macho)

Bibliography

- Levin, Jonathan (September 25, 2019). **OS Internals, Volume 1: User Mode* (v1.3.3.7 ed.). North Castle, NY: Technologeeks. ISBN 978-0-9910555-6-2.
- Singh, Amit (June 19, 2006). *Mac OS X Internals: A Systems Approach*. Addison-Wesley Professional. ISBN 978-0-13-270226-3.

External links

- OS X ABI Mach-O File Format Reference (https://web.archive.org/web/20090819232456/http://developer.apple.com/documentation/DeveloperTools/Conceptual/MachORuntime/index.html) (Apple Inc.)
- `Mach-O(5) (https://keith.github.io/xcode-man-pages/5.Mach-O.html)` – Darwin and macOS File Formats Manual
- Mach Object Files (http://www.cilinder.be/docs/next/NeXTStep/3.3/nd/DevTools/14_MachO/MachO.htmld/index.html) (NEXTSTEP documentation)
- Mach-O Dynamic Library Reference (http://www.fileinfo.com/extension/dylib)
- Mach-O linking and loading tricks (http://blog.darlinghq.org/2018/07/mach-o-linking-and-loading-tricks.html)
- MachOView (https://sourceforge.net/projects/machoview)
- JDasm (https://github.com/Recoskie/JDasm) (Cross-platform disassembler, for macOS, iOS, windows PE, ELF, and file format analysis tool)
